

The Secret Life of a JavaScript String

Understanding Unicode, UTF-16, and the Characters You Thought You Knew

The Web Needed a Universal Language

ASCII/8859-1 Text

A	0100 0001
S	0101 0011
C	0100 0011
I	0100 1001
I	0100 1001
/	0010 1111
8	0011 1000
8	0011 1000
5	0011 0101
9	0011 1001
-	0010 1101
1	0011 0001
	0010 0000
t	0111 0100
e	0110 0101
x	0111 1000
t	0111 0100

Unicode Text

A	0000 0000 0100 0001
S	0000 0000 0101 0011
C	0000 0000 0100 0011
I	0000 0000 0100 1001
I	0000 0000 0100 1001
天	0101 1001 0010 1001
地	0101 0111 0011 0000
س	0000 0000 0010 0000
ل	0000 0110 0011 0011
م	0000 0110 0100 0100
α	0000 0000 0010 0000
ß	0000 0010 0011 1011 0001
γ	0000 0010 0011 1011 0011

The Solution: The Unicode Standard

The Unicode Consortium is the standards body for the internationalization of software and services. Deployed on more than 20 billion devices around the world, Unicode provides the solution for internationalization and the architecture to support localization.

Quick Facts

- Founded in 1988, incorporated in 1991
- Public benefit, 501(c)3 non-profit organization
- Orchestrates the contributions of 100s of professionals, expert volunteers, and language experts
- Funded by membership dues and donations

Every Character Gets a Unique Number: The Code Point

Unicode organizes all characters into charts, assigning each a unique number called a “code point”. This creates a universal map for all the world’s writing systems.

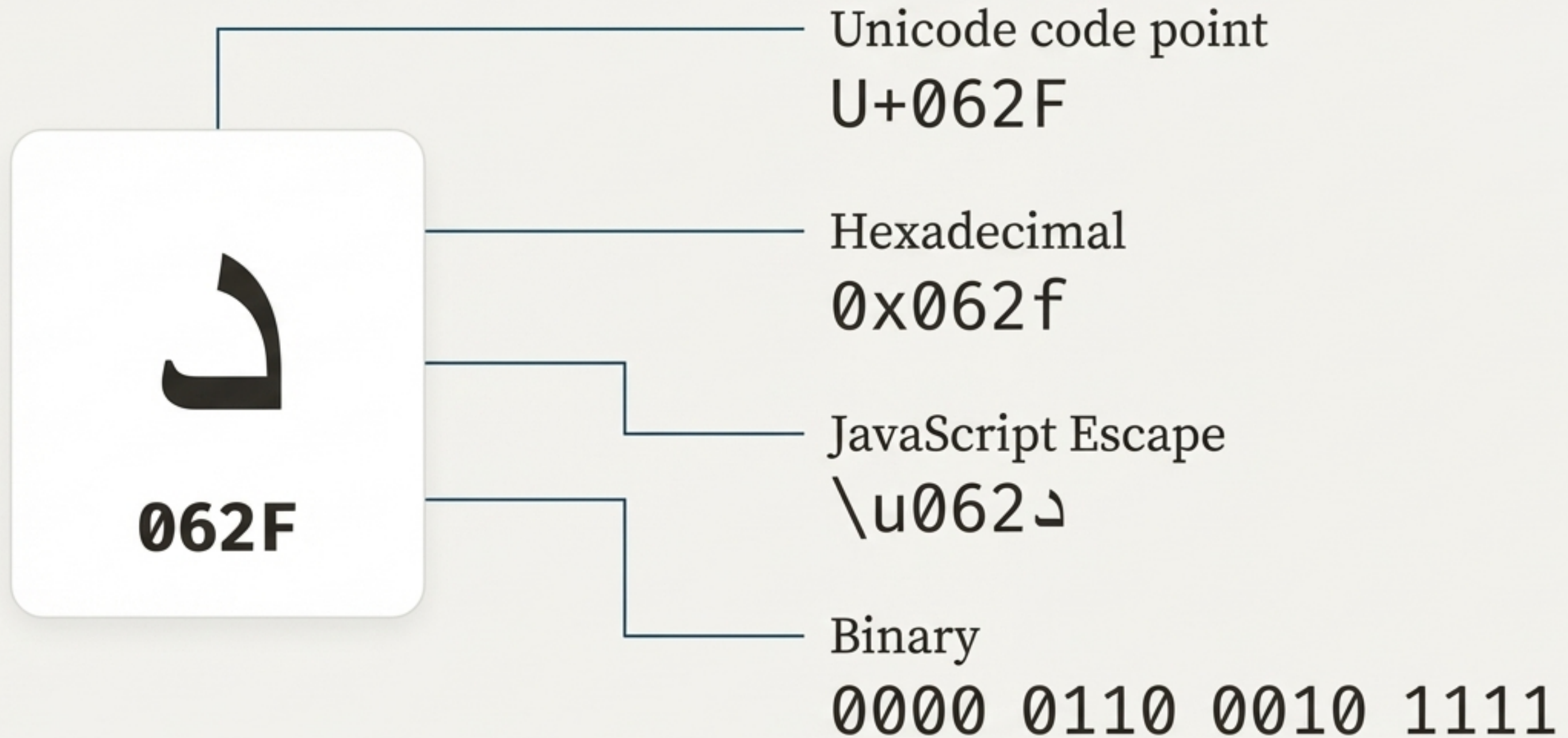
C0 Controls and Basic Latin											
	0	1	2	3	4	5	6	7	8	9	CC
000	CO 0001	!	" 0012	# 0013	% 0014	' 0015	(0016	) 0017	+ 0018	- 0019	/ 0010
001	A 0040	B 0041	C 0042	D 0043	E 0044	F 0045	G 0046	H 0047	I 0048	J 0049	K 004A
002	@ 0040	D 0041	E 0042	F 0043	G 0044	H 0045	I 0046	J 004F	K 0028	L 0029	M 005A
013	O 0048	P 0041	Q 0042	S 00K2	T 0034	U 00H5	V 0056	W 0057	X 0038	Y 0039	Z 005A
014	! 0019	 0018	~ 0017								
015	{ 0021	} 0022	~ 0023								

Code Point: U+0041
Latin capital letter 'A'.
Assigned a unique
universal number.

Arabic											
	0620	0622	0624	0626	0628	062C	0624	0622	0626	0626	
0620								ة 0616	ه 0617	ا 0618	
0622	ز 0E26	د 0G28	د 0627	ب 0628	ح 0629	ح 0620	ژ 0625	ط 0621	ظ 0622	م 0622	
0624	ث 0625	ث 0626	ء 0027	ب 0028	ق 0621	ش 0620	ع 0625	ظ 0620	ط 0621	ك 0623	
0626	ل 0623	ه 0624	ء 0624								
0628	ي 0630										

Code Point: U+0628
Arabic letter 'Beh'.
Assigned a unique
universal number.

Anatomy of a Code Point



How JavaScript Stores Strings: UTF-16 Encoding

JavaScript engines represent strings internally using UTF-16. This encoding uses a minimum of 16 bits (two bytes) for every code point.

A = 00000000 01000001

16 bits per character



The 16-Bit Question

Is 65,536 Characters Enough?



With 16 bits, we can represent 2^{16} , or 65,536 unique characters. This range (from U+0000 to U+FFFF) is called the Basic Multilingual Plane (BMP).



*But what about the vast number of characters beyond the BMP?
Think ancient scripts, complex symbols, and... emojis.*



Life Beyond the Basic Multilingual Plane

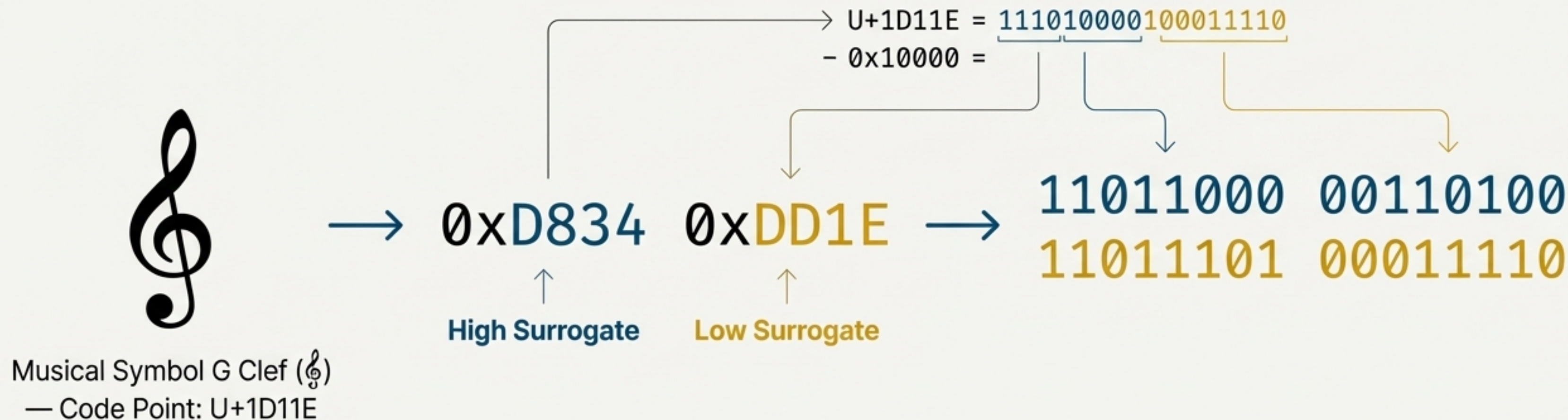
Many characters exist with code points higher than U+FFFF.

Egyptian Hieroglyphs								
U+13000 range								
3040		U+13050	U+13060	U+13070	U+13080	U+13090	U+13070	U+13013
cael 360	0	 13050	 13060	 13070	 13080	 shoil 13055	 kam 13070	 bora 13081
oan 051	1	 13051	 13061	 13071	 13081	 biamoad 13051	 neak 13062	 amve 13081
ge 380	2	 13052	 13062	 13072	 13082	 U+13063 13063	 gaun 13062	 mone 13085
eit 073	3	 wan 13055	 cohn 13066	 IFOR 13077	 den 13078	 sitr 13079	 fst 13071	 green 13072
kes	4	 poond	 bast	 voeus	 kotéo	 niite aarth	 verta	 zili

Emoticons					
U+1F600 range					
	U+1F600	U+1F610	U+1F620	U+1F630	U+1F640
0	 U+1F600	 U+1F610	 U+1F620	 U+1F630	 U+1F640
1	 U1F601	 U1F611	 U1F621	 U1F631	 U1F641
2	 U1F602	 U1F612	 U1F622	 U1F632	 U1F642
3	 U1F603	 U1F613	 U1F623	 U1F633	 U1F643
					

The Solution: UTF-16 Surrogate Pairs

To represent characters beyond the BMP, UTF-16 uses a pair of 16-bit code units called a surrogate pair. A reserved range of code points (0xD800 to 0xDFFF) is used for this.

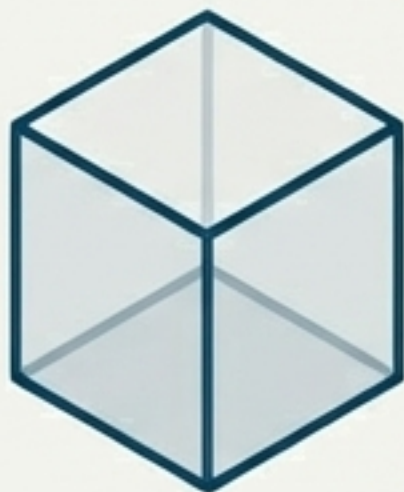


The Practical Implication: Why '🎵'.length is 2

JavaScript's `.length` property does not count characters. It counts the number of 16-bit UTF-16 code units in the string.

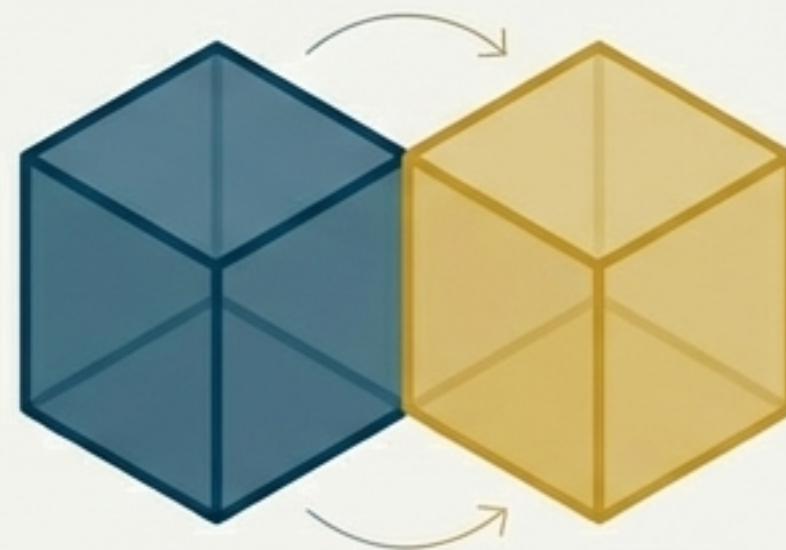
Single-unit character

```
const letter = 'A'; // 1 code unit  
console.log(letter.length); // Output: 1
```



Surrogate pair character

```
const clef = '🎵'; // 2 code units (surrogate pair)  
console.log(clef.length); // Output: 2
```



How to Iterate Over Strings Correctly

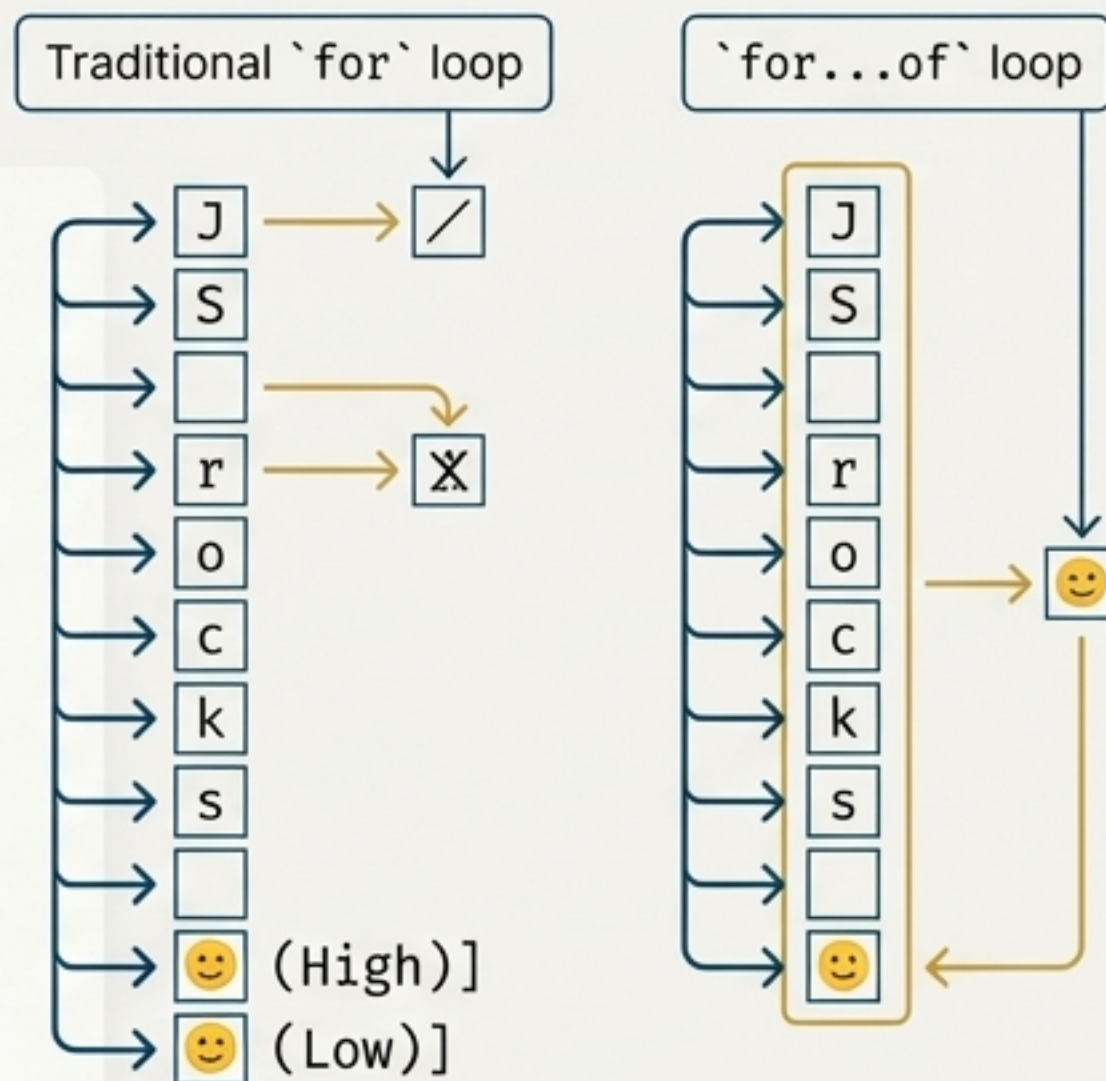
A standard `for` loop can incorrectly split surrogate pairs. Modern JavaScript offers safer ways to iterate by code point.

The Correct Way

```
const emojiString = 'JS rocks 🌟';

// The for...of loop correctly handles the emoji
for (const char of emojiString) {
  console.log(char);
}
// Output: J, S,   , r, o, c, k, s,   , 🌟

// To get the true character count:
console.log(Array.from(emojiString).length); // Output: 10
```



Visualizing a String in Memory

Text: “Cloud Native”

Hexadecimal Representation

Hexadecimal

\u0043	\u006C	\u006F	\u0075	\u0064	\u0020	\u004E	\u0061	\u0074	\u0069	\u0076	\u0065
--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------

Binary Representation

Binary

0000 0000	0100 0011	0000 0000	0110 1100	0000 0000	0110 1111	0000 0000	0111 0101
0000 0000	0110 0100	0000 0000	0010 0000	0000 0000	0100 1110	0000 0000	0110 0001
0000 0000	0111 0100	0000 0000	0110 1001	0000 0000	0111 0110	0000 0000	0110 0101

The Unicode Encoding Family: A Comparison

UTF-16 is just one way to encode Unicode. UTF-8 and UTF-32 are other common standards with different trade-offs.

UTF-8	<code>0xE1 0xBA 0x9E</code> Fira Code		Variable width, 3 bytes for this char. Dominant on the web for its ASCII efficiency. Source Serif Pro
UTF-16	<code>0x1E9E</code>		2 bytes for this char. Used by JavaScript. Source Serif Pro
UTF-32	<code>0x0001E9E</code>		Fixed width, always 4 bytes. Simple but memory-intensive. Source Serif Pro

Key Takeaways for JavaScript Developers



- JavaScript strings are sequences of **UTF-16 code units**, not characters. This is the fundamental mental model.



- `.length` counts code units. For strings with emoji or other complex characters, this is not the **same as the character count**.



- Characters outside the Basic Multilingual Plane (BMP) **require two code units (a surrogate pair)**.



- Always use `for...of` or `Array.from()` for safe, **character-aware** iteration and counting.

From Limited ASCII to a Universal Language

Understanding how strings work under the hood is more than a technical exercise. It allows us to build software that correctly handles the richness and diversity of human language, ensuring our applications work for everyone, everywhere